NBA Play Ranker Software Design Document

INFO45828 Computer Science Capstone Project
Alpha Release
Gurkaranjit Asahan and Abdul Mousa

Contents

Introduction....4

System architecture....4

- Presentation....4
- Application....4
- Domain....4
- Infrastructure....4
- 1. Software System Architecture....5
- 2. Software System Architecture - Subsystem Dependencies....7

Deployment....8

- 3. Deployment Diagram....9

Alpha collaboration designs....10

- Compare Usage and Efficiency....10
- 4. Collaboration Overview - Compare Usage and Efficiency....11
- 5. Structural Design - Compare Usage and Efficiency....12
- 6. Detailed Sequence - Compare Usage and Efficiency....13
- View Opponent Defensive Weak Spots....13
- 7. Collaboration Overview - View Opponent Defensive Weak Spots....14
- 8. Structural Design - View Opponent Defensive Weak Spots....15
- 9. Detailed Sequence - View Opponent Defensive Weak Spots....16

Alpha screen flow....17

- 10. Wireflow - Baseline Comparisons....17

Existing collaboration designs....18

- Analyst workspace....18
- 11. Collaboration Overview - Access Analyst Workspace....18
- 12. Structural Overview - Access Analyst Workspace....19
- 13. Behavioural Overview - Access Analyst Workspace....20
- Baseline recommendations....22

14. Collaboration Overview - Ranked List via API (Baseline)....22
15. Structural Overview - Ranked List via API (Baseline)....23
16. Behavioural Overview - Ranked List via API (Baseline)....24
Context recommendations....26
17. Collaboration Overview - Rank Plays with Game Context....26
18. Structural Overview - Rank Plays with Game Context....27
19. Behavioural Overview - Rank Plays with Game Context....28
Authentication and access....30
20. Collaboration Overview - Authenticate Analyst Access....30
21. Structural Overview - Authenticate Analyst Access....31
22. Behavioural Overview - Authenticate Analyst Access....32
23. Collaboration Overview - Validate Coach Access....34
24. Structural Overview - Validate Coach Access....35
25. Behavioural Overview - Validate Coach Access....36
Shared result exports....38
26. Collaboration Overview - Export KPIs / Reports (486)....38
27. Elaboration - Shared Export Controls (486)....39
28. Detailed Sequence - Export Displayed Results (486)....40
Source and request contracts....41
Release boundaries....41

Introduction

The application separates browser screens, API workflows, basketball calculations and infrastructure. This design describes the current deployment and the Alpha comparison path from the coach's filter selection to validated source totals and chart data. Each detailed use case links to its collaboration overview, structural design and sequence diagram.

System architecture

Presentation

Next.js and React display the coach and analyst workspaces. MatchupDashboardClient holds the selected season, teams and possession cutoff. UsageEfficiencyChart and OpponentDefenseChart render accessible charts with exact values in tables. Changing filters removes the previous results immediately.

Application

FastAPI coordinates requests, checks identity and permissions, validates filters and invokes the domain service. GET /analytics/playtypes supplies both Alpha views from one consistent response. Invalid inputs, unavailable data and expired sessions return clear errors rather than partial comparisons.

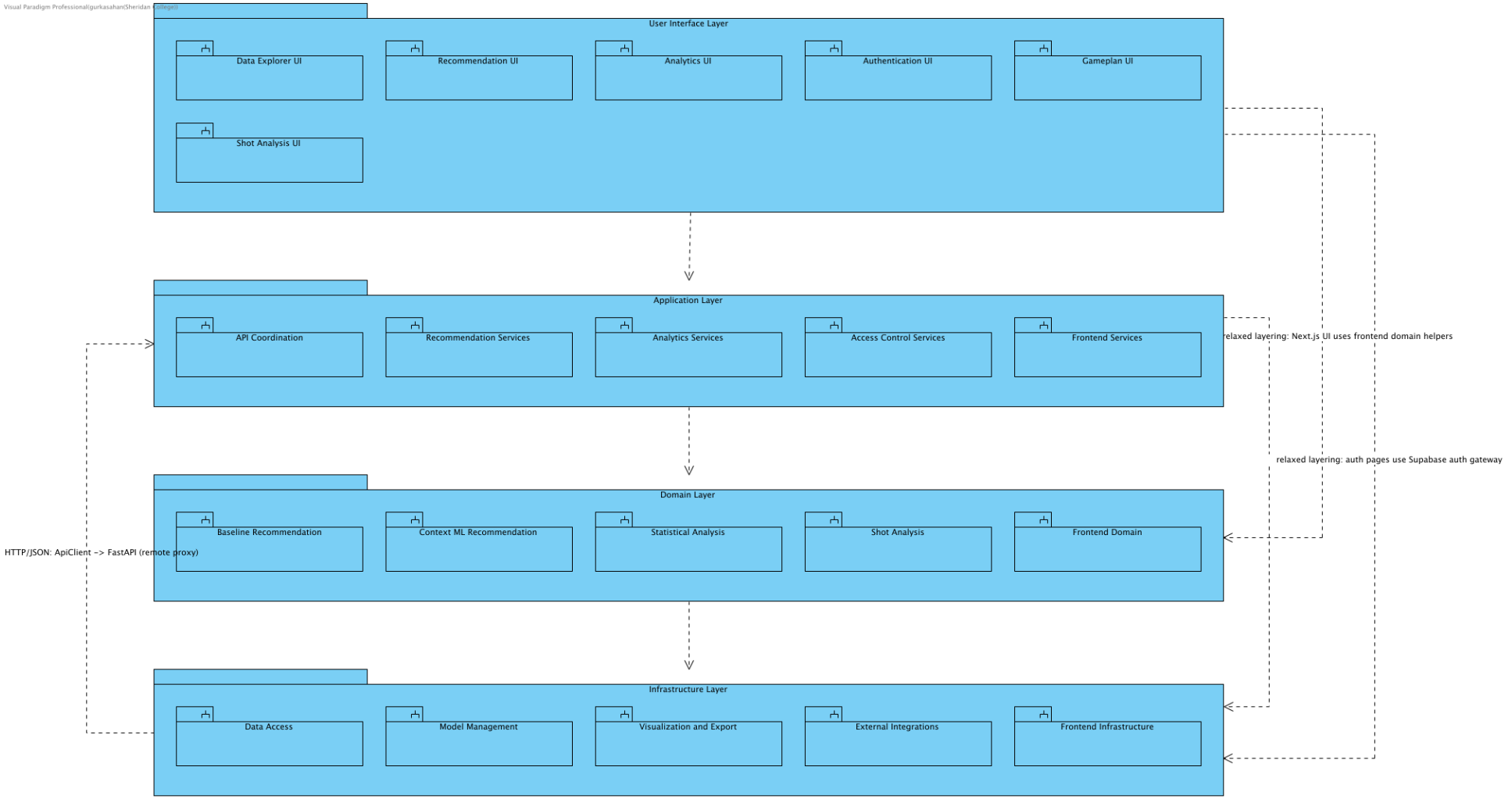
Domain

The baseline service aggregates regular-season play-type records. The dashboard calculates usage, points per possession and differences from the league for the same play type. The existing baseline recommender blends team offense with opponent points allowed. Model and shot services remain separate from the new descriptive dashboard.

Infrastructure

Data access loads versioned source files and validated cached artifacts. The frontend request adapter attaches the current session token, validates the response and cancels obsolete requests. Export services handle existing CSV, JSON and PDF reports. Source and aggregation versions identify the inputs behind saved predictions.

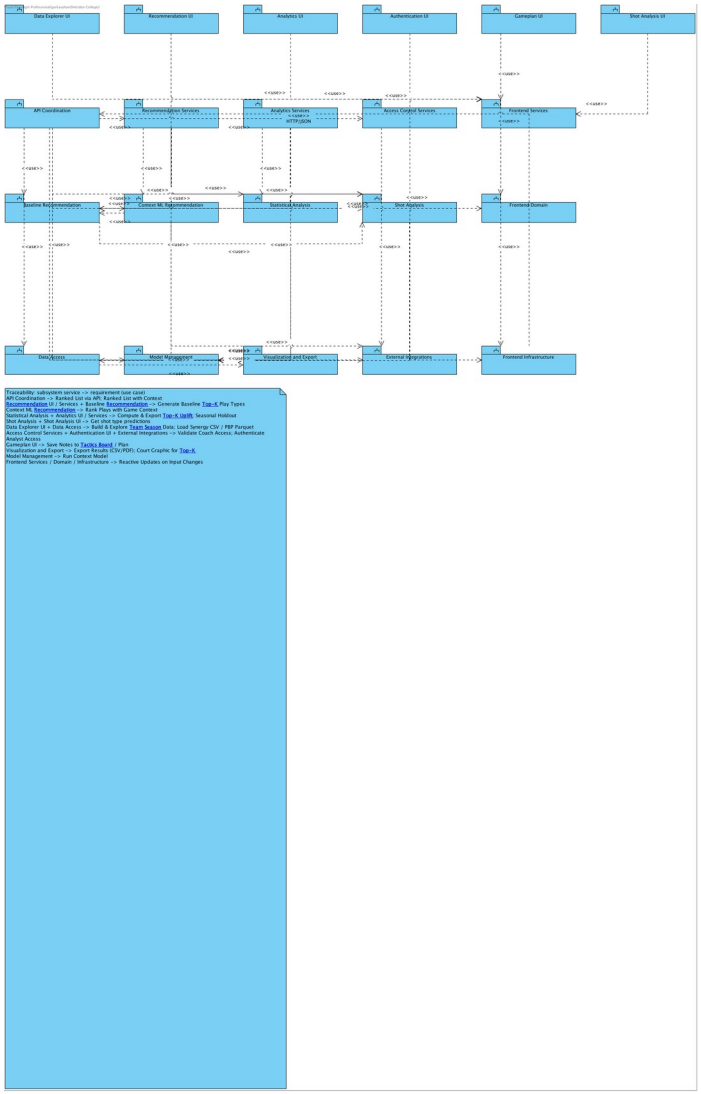
1. Software System Architecture



System type: NBA Play Ranker is a web-based decision-support information system. Coaches and analysts select a matchup (season, our team, opponent) and optionally a game situation, and the system returns a ranked Top-K list of offensive play types with explanations, visual evidence (court zone and shot heatmaps) and CSV/PDF exports.Architecture: a four-layer architecture (User Interface -> Application -> Domain -> Infrastructure) implemented across two deployable tiers: a Next.js/React frontend (Vercel) and a Python FastAPI backend, with Supabase providing authentication and user roles. Each layer is decomposed into subsystems; each subsystem exposes public interfaces (interfaces.py Protocol/ABC classes in the backend, interfaces.ts in the frontend, or the Next.js route contract for UI pages) and hides its implementation modules.Architectural decisions

and rationale:1. Layered architecture instead of microservices or pipe-and-filter: the system has one small team, one dataset family and one request/response workflow (select matchup -> rank -> explain -> export). Layers give a clear separation of concerns and a strict dependency direction without the operational cost of many services; the offline data pipeline (backend/data/etl) is kept outside the request path, so the online system only serves pre-built, cached aggregates and stored ML predictions.2. Recommendation logic lives only in the Domain layer (Baseline Recommendation, Context ML Recommendation). The baseline recommender is the explainable trust anchor; the context ML recommender produces adjusted rankings and reports an explicit error if its model data is unavailable, so ML can be changed without touching the UI or the API.3. The API Coordination subsystem is the single entry point (Facade) for the UI: FastAPI routers validate inputs, enforce roles through dependency injection (Depends(require_auth / require_role)) and delegate to domain and infrastructure subsystems.4. Security is centralised in Access Control Services (role -> resource permission table) and External Integrations (Supabase JWT verification), and mirrored in the Next.js middleware for page routes.5. Infrastructure subsystems isolate technical concerns: Data Access (parquet loading, fingerprint-based cache, Repository-style accessors), Model Management (cross-validated model evaluation), Visualization and Export (SportyPy renderers, PDF builder), External Integrations (Adapter for Supabase and spaCy NLP), Frontend Infrastructure (ApiClient remote proxy over HTTP/JSON).6. Frontend relaxed layering: UI components may call pure frontend domain helpers (app/domain) and the Supabase auth gateway directly; all backend data access goes through the app/services facades and the ApiClient.Quality attributes supported: maintainability and testability (subsystem interfaces, one module per concern), performance (cached aggregates, precomputed predictions, lru/disk caches), security (JWT + role checks at both tiers), explainability (rationale produced next to every ranking) and explicit failure handling (Context reports unavailable model data; the separate baseline page remains available).Note: the offline ETL scripts in backend/data/etl (build_pbp_pipeline, pbp_phase2_build, run_ml_evaluation) are build-time tooling and are not part of the runtime layers. The root Next.js shell (app/layout.tsx, app/page.tsx home landing) is presentation-only and is not modelled as a subsystem.

2. Software System Architecture - Subsystem Dependencies

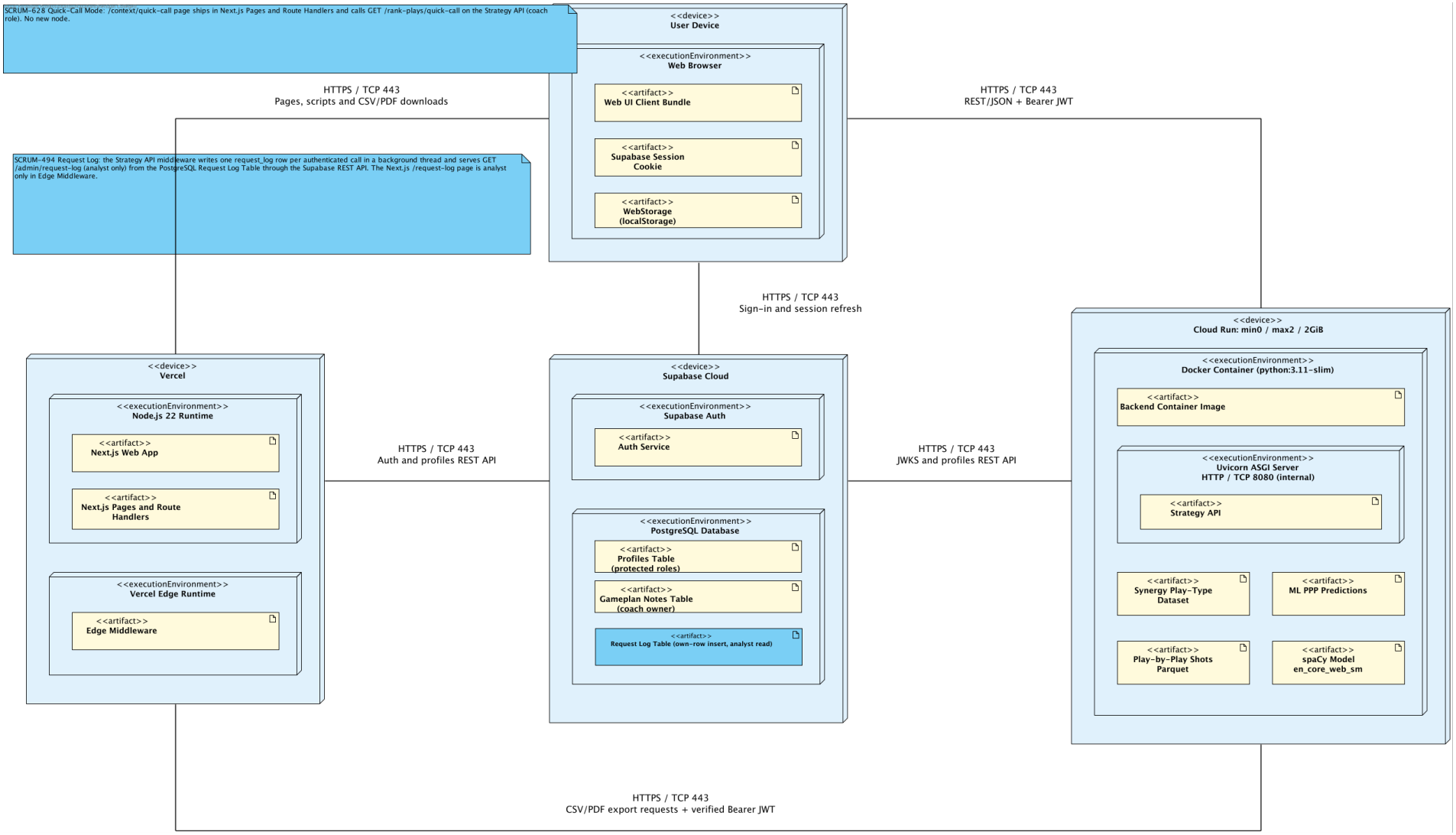


Dependency (USES) structure between Design Model subsystems, derived from the import statements of the NBAPlayRanker-1 code base (backend Python packages and Next.js app/ modules). Each arrow means the client subsystem imports/calls the supplier subsystem.

Deployment

The browser reaches the Next.js frontend on Vercel and the FastAPI service on Google Cloud Run over HTTPS on port 443. Supabase provides authentication and PostgreSQL-backed application storage. The Cloud Run container listens on its configured internal port. Offline scripts prepare model and analysis artifacts; normal dashboard requests do not train models. The existing deployment diagram records the service boundaries and labelled protocols.

3. Deployment Diagram



SCRUM-506. Frontend https://nbaplayranker-seven.vercel.app on Vercel Hobby, Node.js 22, manually deployed from canonical shared Git checkout. Cloud Run nba-playranker-api in northamerica-northeast1: Python3.11/Uvicorn application.api_coordination.app:app, 1CPU/2GiB, min0/max2, concurrency2, numeric threads1. HTTPS REST/JSON + Bearer and explicit frontend CORS. Supabase Free Auth/JWKS, protected profiles and coach-owned gameplan_notes RLS. Analyst uplift/calibration reports use packaged datasets and ephemeral instance cache. Coach Gameplan notes,

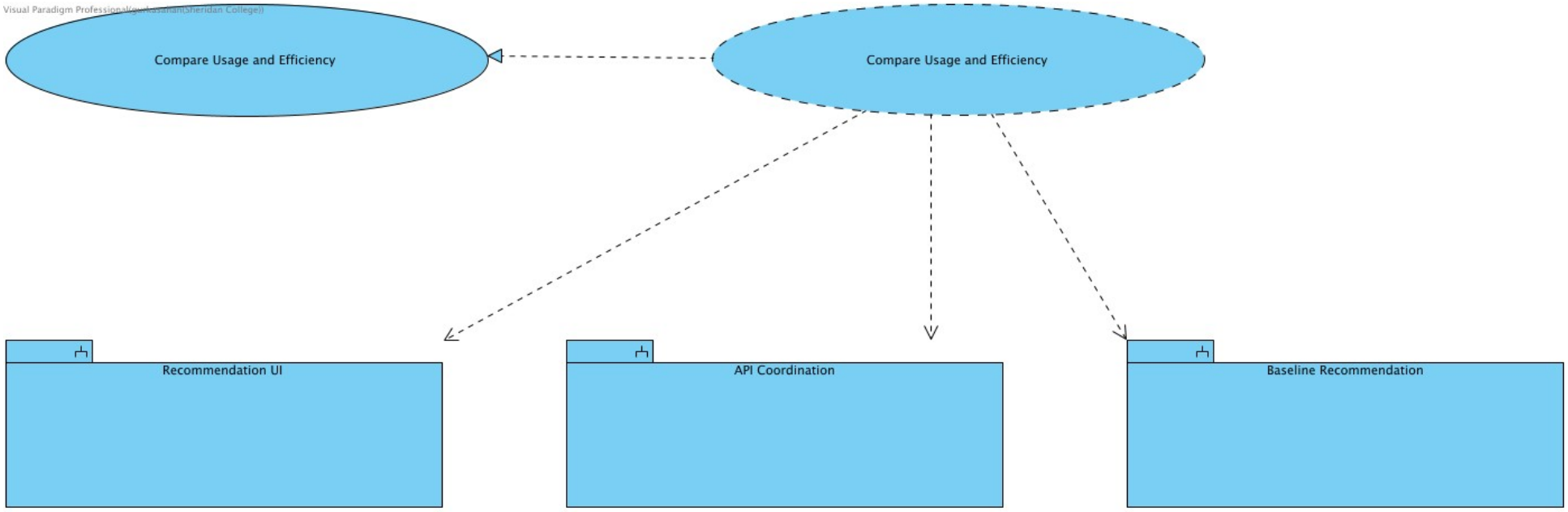
plans and roles synchronize with the coach-owned Supabase table through authenticated owner-only REST requests. CAD10 budget alerts and CAD7 Cloud Run cap have delayed enforcement, not an absolute total billing guarantee.Production authorization validates ES256/RS256 JWT signatures against Supabase JWKS plus issuer, audience and expiry; verified subject identifies the user. The current protected public.profiles.role authorizes resources, never user_metadata.role. Unassigned accounts remain pending. Analyst evidence endpoints require analyst access; coach recommendations and the matchup dashboard require coach access. Shared shot views accept either approved role. Invalid tokens return 401 and forbidden roles return 403.Deployment view: artifacts are grouped inside their execution environments. Six host communication paths show HTTPS on TCP port 443. Cloud Run terminates TLS and forwards HTTP to Uvicorn on internal port 8080. Supabase access uses HTTPS Auth and REST APIs; no direct PostgreSQL connection is used by the app. Browser Gameplan state synchronizes with Supabase using INSERT for a new matchup and revision-filtered updates; local draft recovery remains available for conflicts or interrupted edits. Detailed deploy, composition and artifact dependency relationships remain in the model and are hidden only on this view to avoid duplicate arrows.

Alpha collaboration designs

Compare Usage and Efficiency

Use case A35cd925e814ba68; SCRUM-695. The collaboration reuses the existing application and deployment layers. Numeric fields and alternate outcomes follow the SRS Flow of Events.

4. Collaboration Overview - Compare Usage and Efficiency



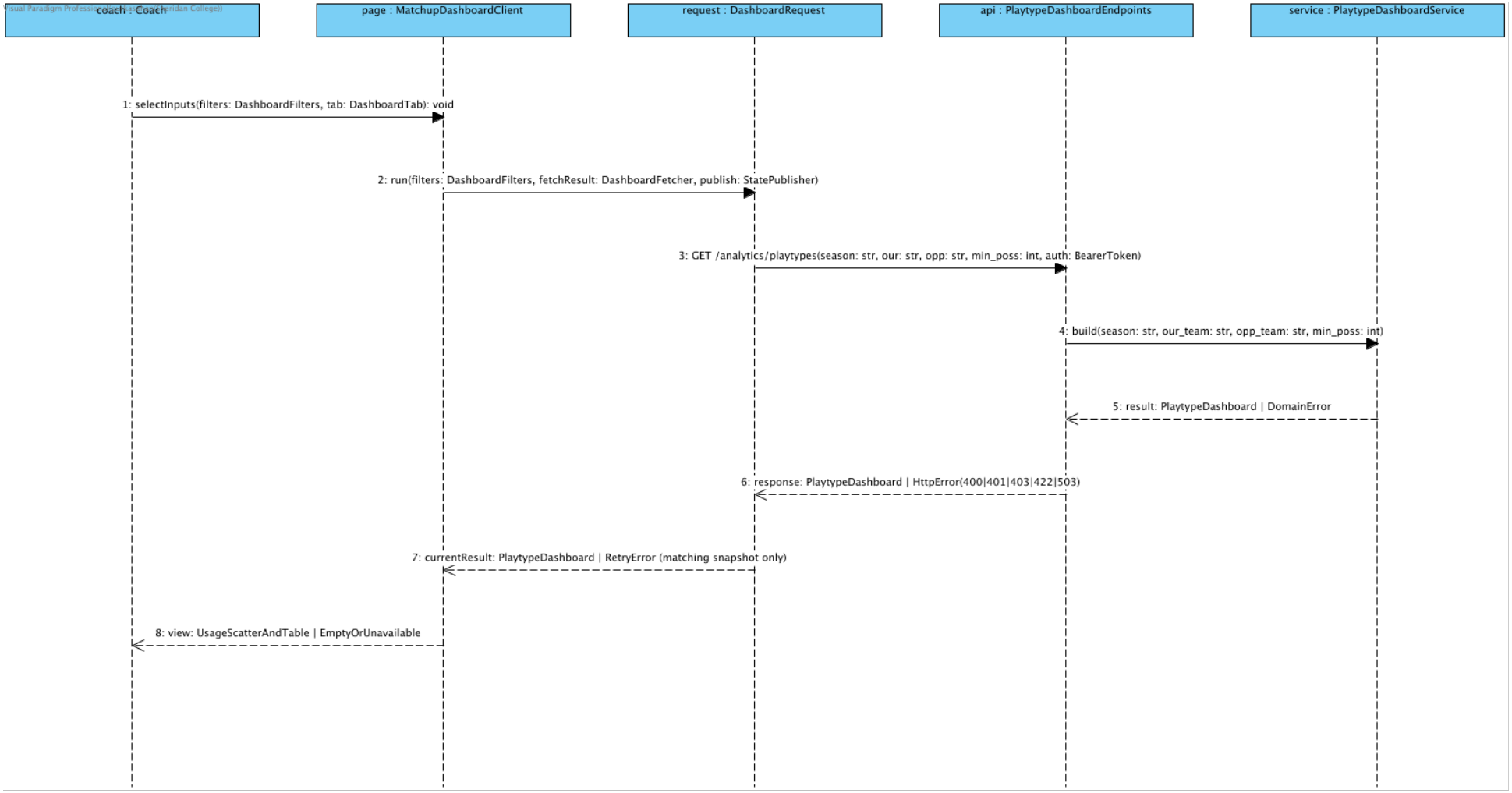
Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

5. Structural Design - Compare Usage and Efficiency



Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

6. Detailed Sequence - Compare Usage and Efficiency

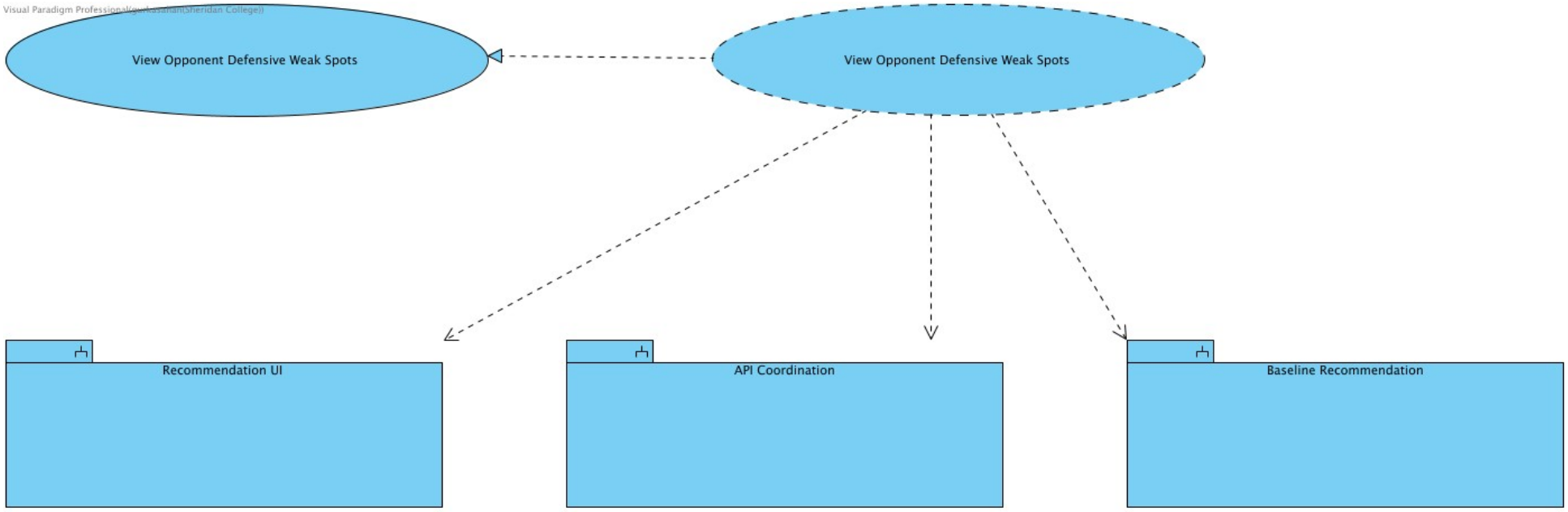


Detailed design for Compare Usage and Efficiency. Historical usage/PPP relative to league. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

View Opponent Defensive Weak Spots

Use case A308531b428a7bd1; SCRUM-696. The collaboration reuses the existing application and deployment layers. Numeric fields and alternate outcomes follow the SRS Flow of Events.

7. Collaboration Overview - View Opponent Defensive Weak Spots



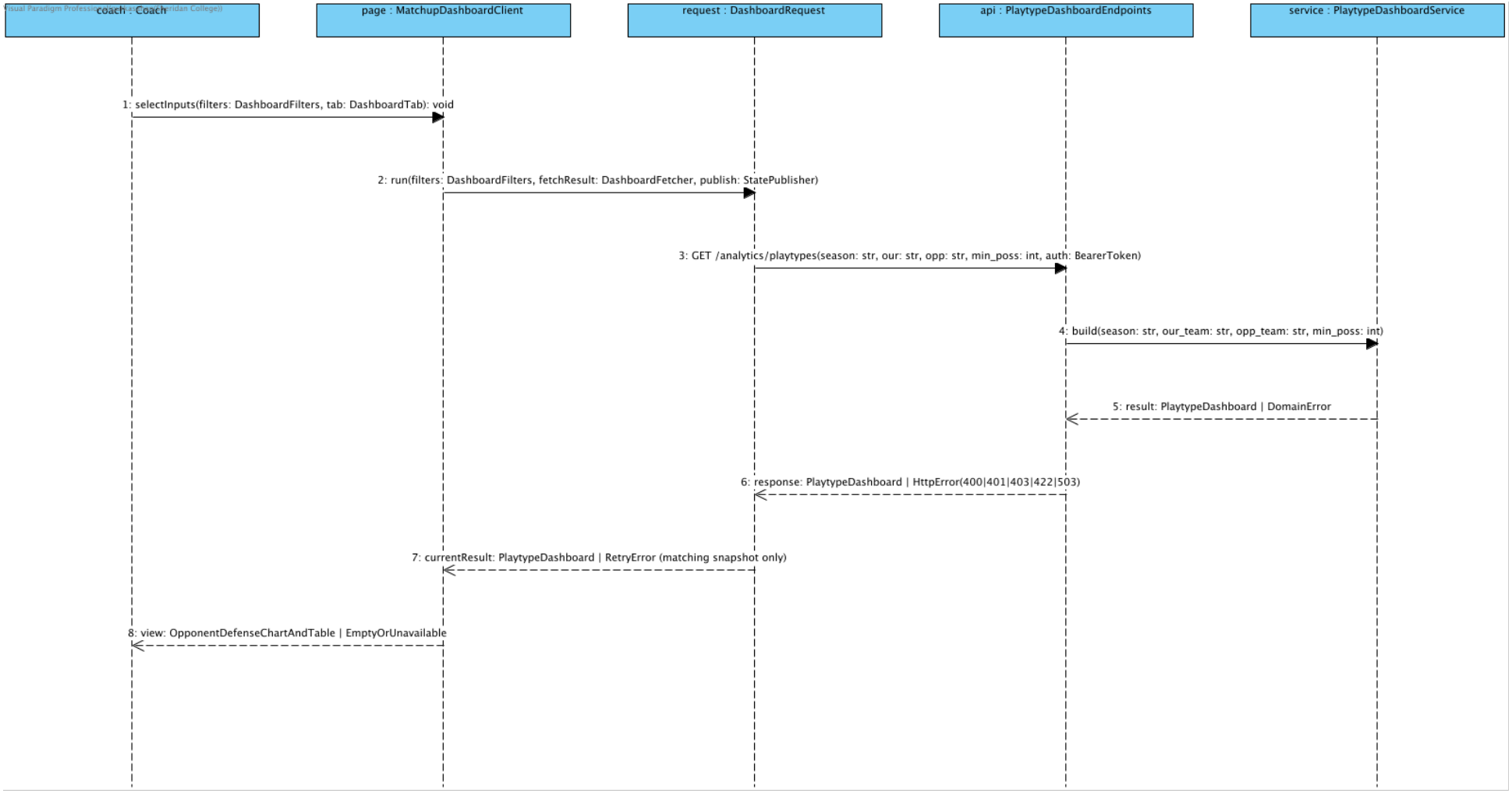
Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

8. Structural Design - View Opponent Defensive Weak Spots



Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

9. Detailed Sequence - View Opponent Defensive Weak Spots



Detailed design for View Opponent Defensive Weak Spots. Opponent allowed PPP relative to league; positive sufficiently sampled differences only. The API verifies coach access and source quality before invoking service; alternate outcomes are defined in Flow of Events. Existing source/model and deployment components are reused.

Alpha screen flow

The native wireflow describes conceptual screen states. It is a model of navigation and results, not a screenshot of the deployed application. Valid filter changes start a request; view changes reuse the current response.

10. Wireflow - Baseline Comparisons

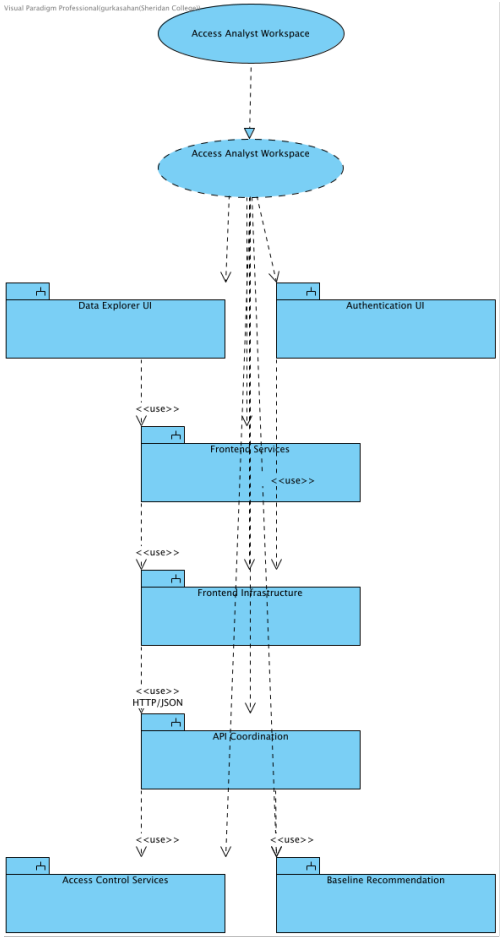


SCRUM-695/696. Native conceptual screen-state design for the implemented coach /matchup-dashboard. These are model views, not captured application screenshots. Valid filters automatically load both views; switching views needs no new request. Beta and Final requirements remain planned.

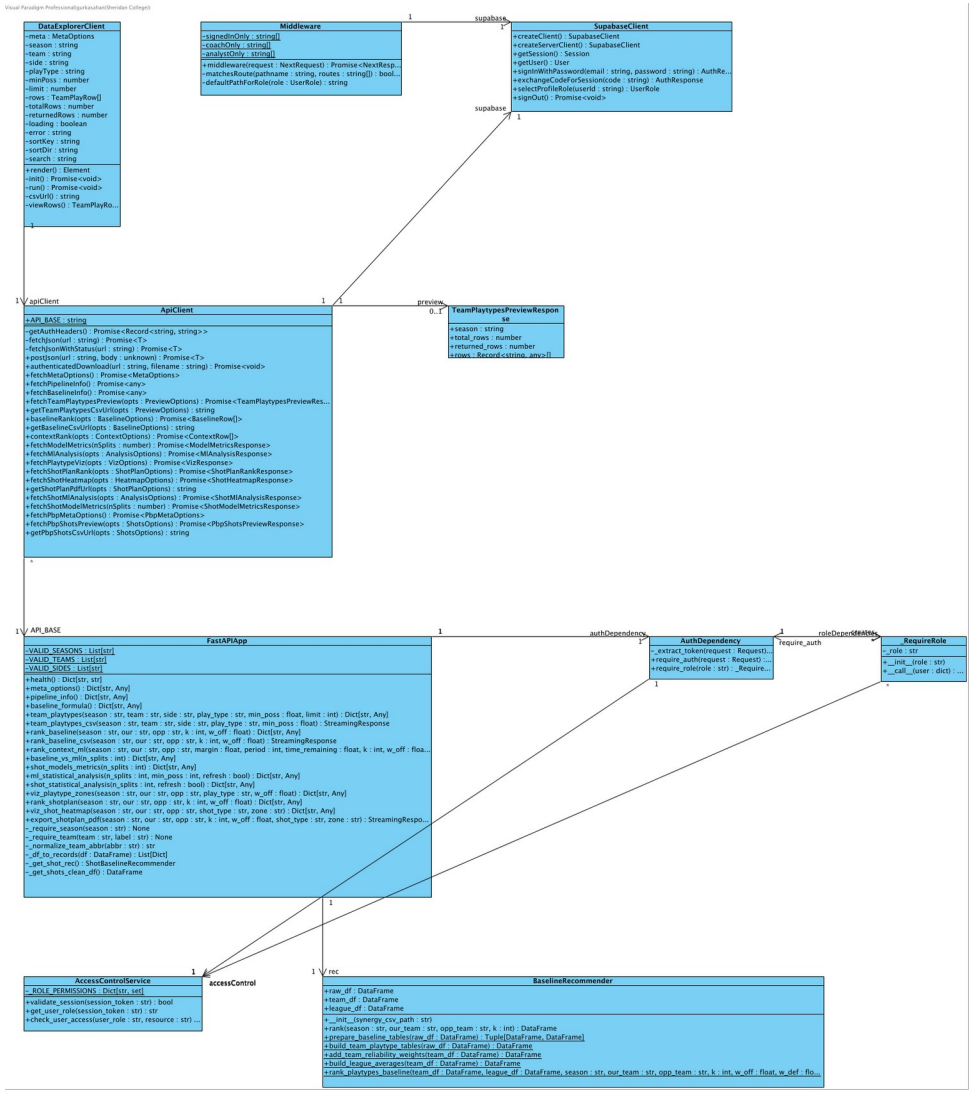
Existing collaboration designs

Analyst workspace

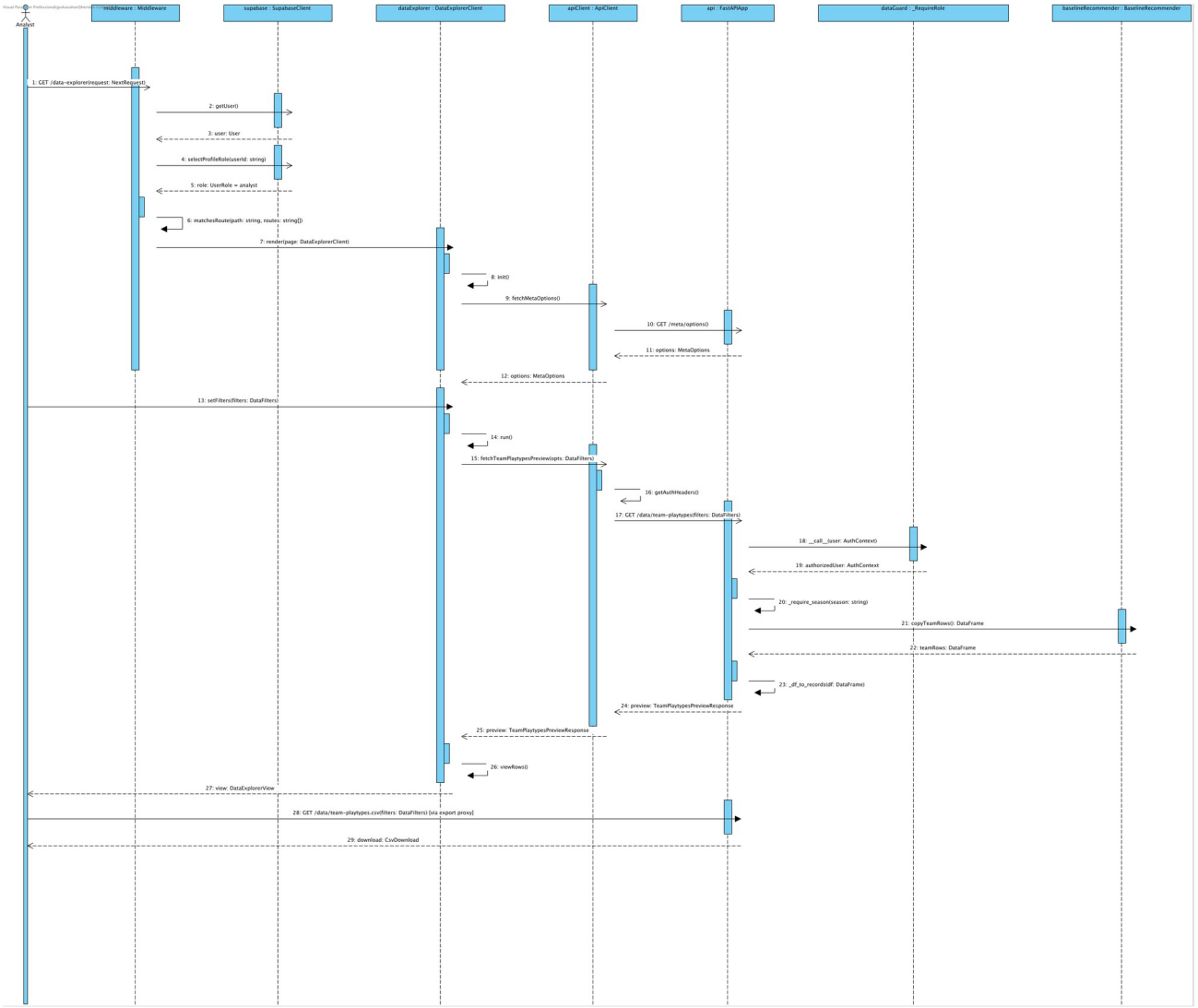
11. Collaboration Overview - Access Analyst Workspace



12. Structural Overview - Access Analyst Workspace



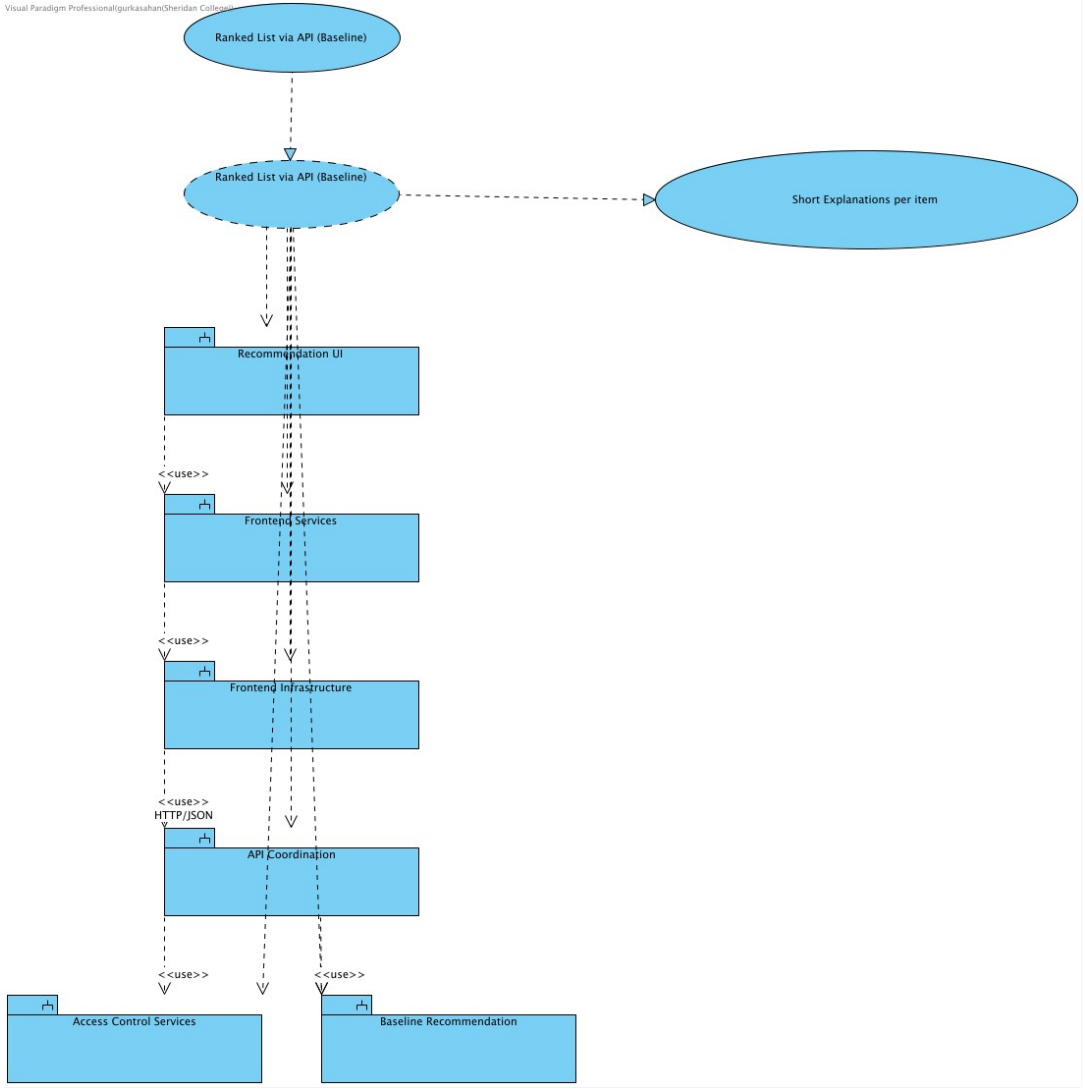
13. Behavioural Overview - Access Analyst Workspace



Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.
Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
DataFilters = {season: string, team?: string, side?: offense | defense, play_type?: string, min_poss: number >= 0, limit: integer 1..2000}; preview defaults limit=200. CSV uses the same filters except limit. TeamPlaytypesPreviewResponse = {season: string, total_rows: integer, returned_rows: integer, rows: TeamPlayRow[]}. TeamPlayRow contains the selected source columns, including PLAY_TYPE, POSS and PPP.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
CsvDownload = {status: 200, mediaType: text/csv, filename: string, body: UTF-8 bytes}. Credentials and concrete JWT values are intentionally absent. Promise completion returns the named payload; failures reject or return the labeled HTTP error and do not become successful exports.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

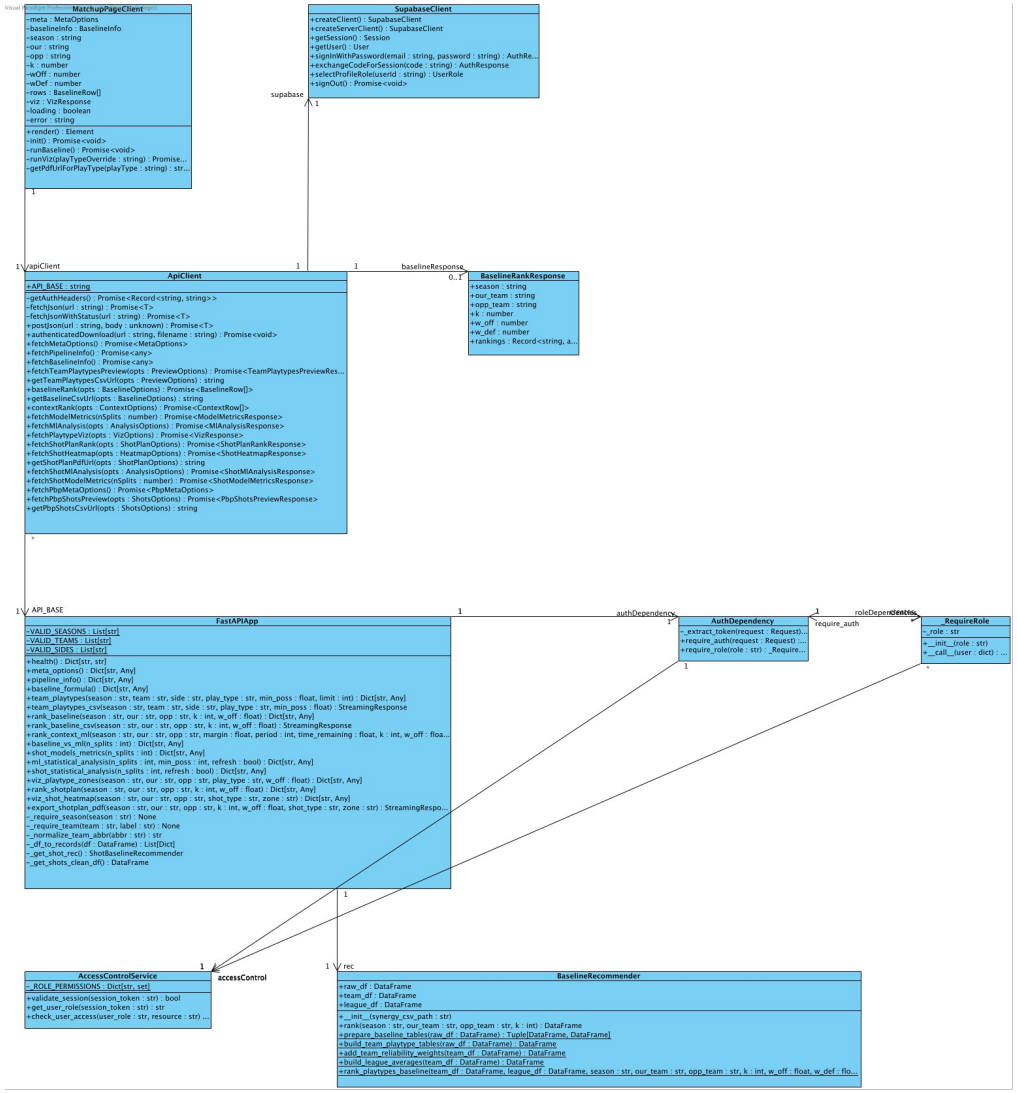
Baseline recommendations

14. Collaboration Overview - Ranked List via API (Baseline)

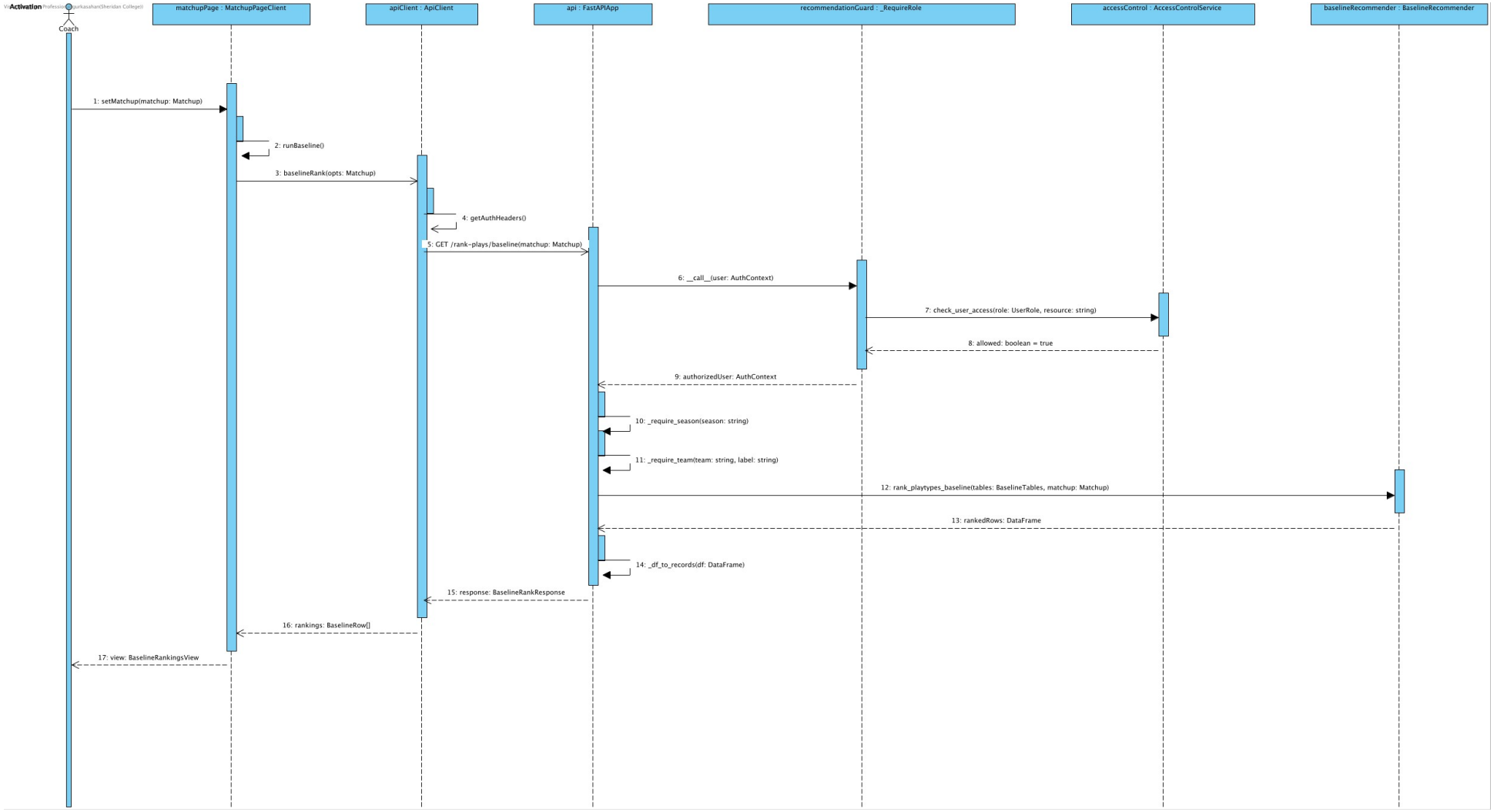


SCRUM-546: this collaboration also supplies short explanations for its own current ranking. Baseline explains season aggregates and unused score/period/clock; context identifies the conditions actually applied. No causal or future-accuracy guarantee is added.

15. Structural Overview - Ranked List via API (Baseline)



16. Behavioural Overview - Ranked List via API (Baseline)

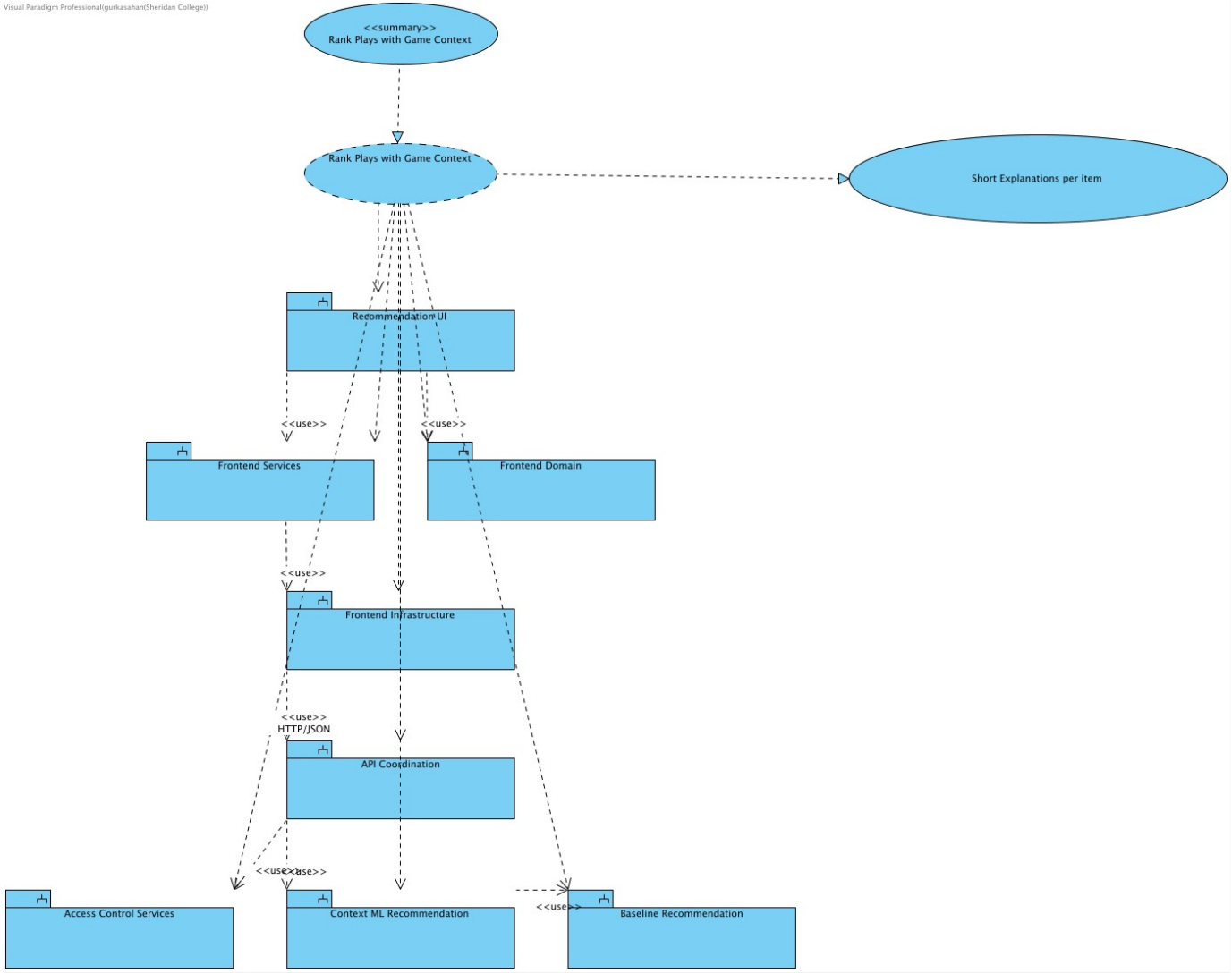


Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.

Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

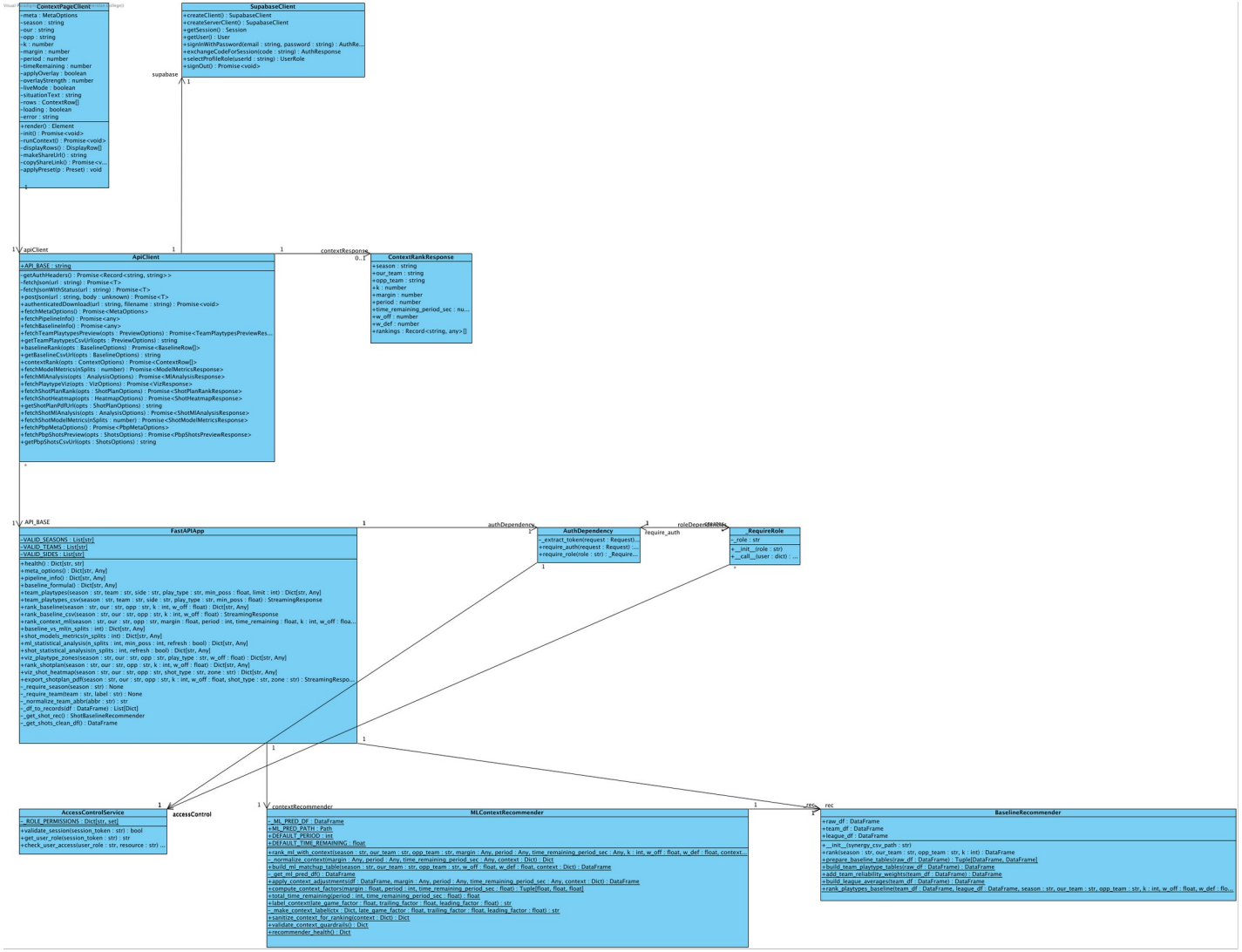
Context recommendations

17. Collaboration Overview - Rank Plays with Game Context

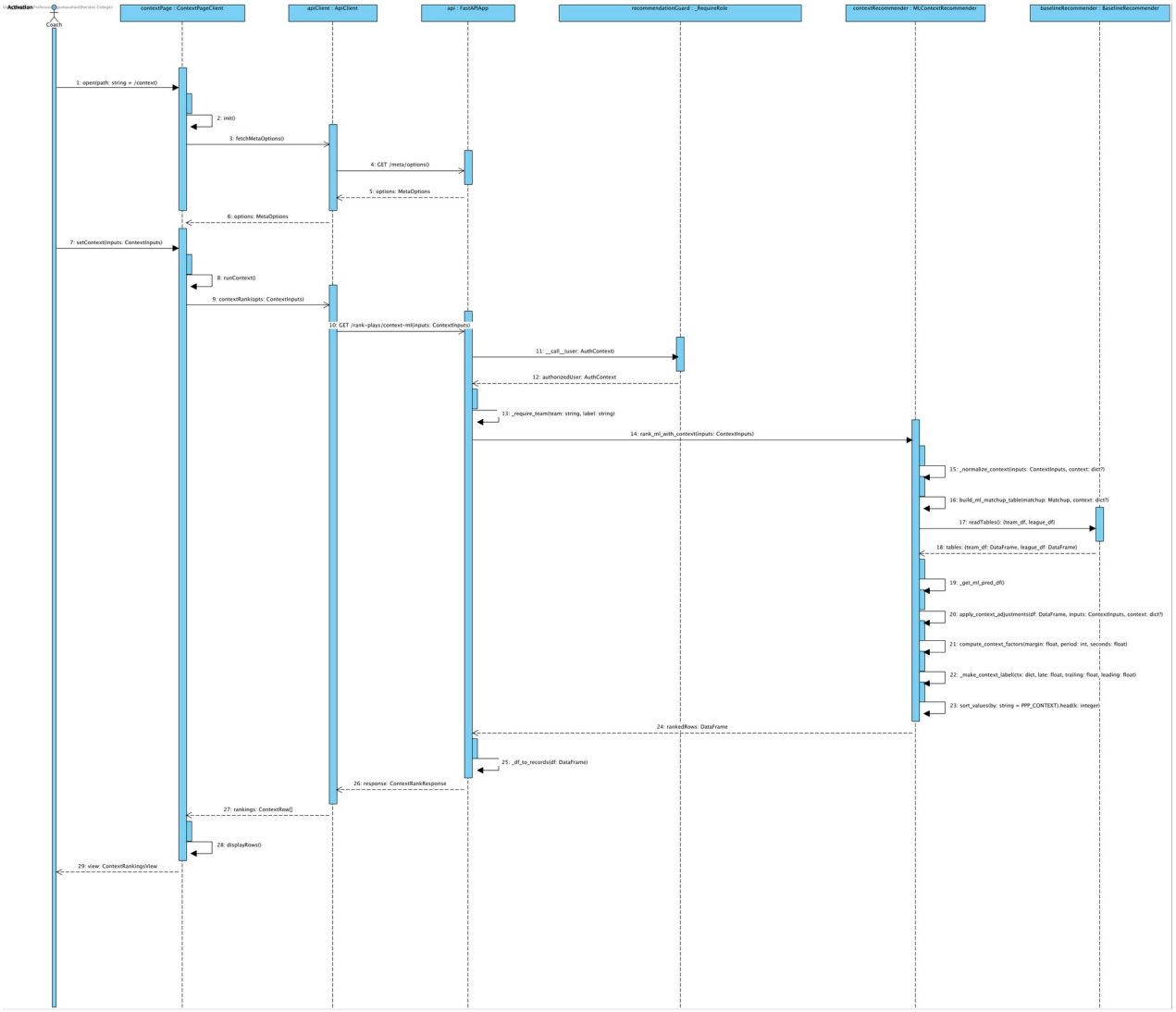


SCRUM-546: this collaboration also supplies short explanations for its own current ranking. Baseline explains season aggregates and unused score/period/clock; context identifies the conditions actually applied. No causal or future-accuracy guarantee is added.

18. Structural Overview - Rank Plays with Game Context



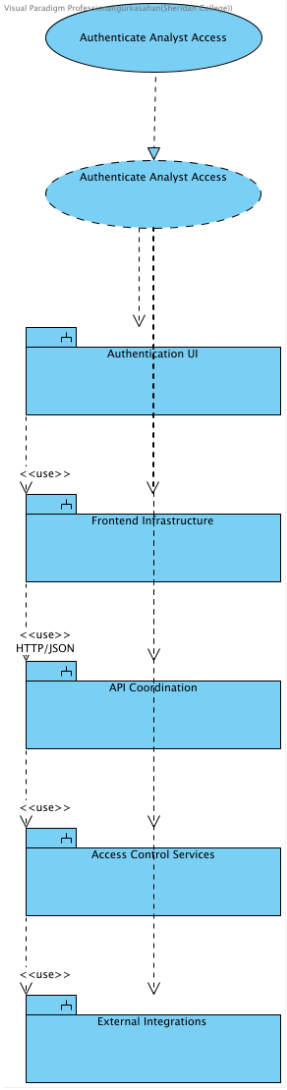
19. Behavioural Overview - Rank Plays with Game Context



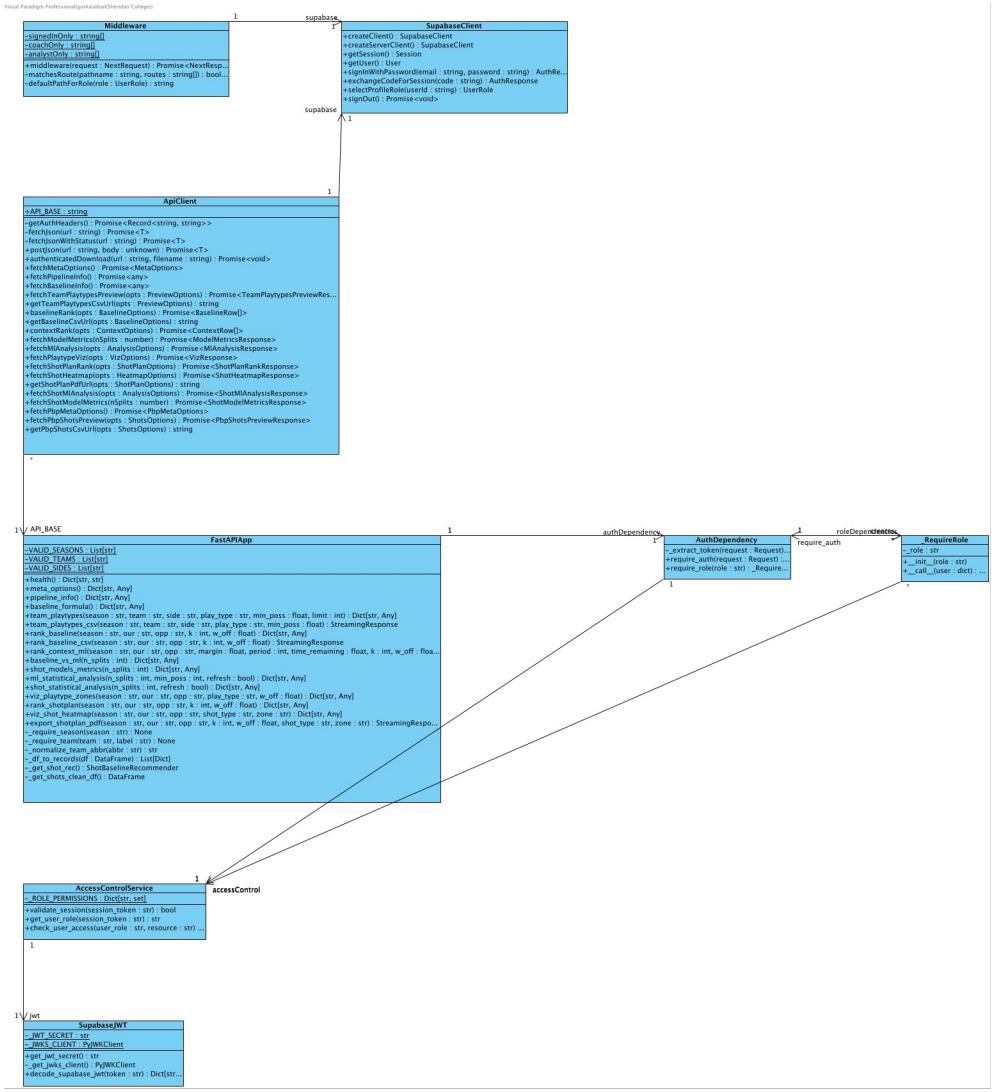
Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.
Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

Authentication and access

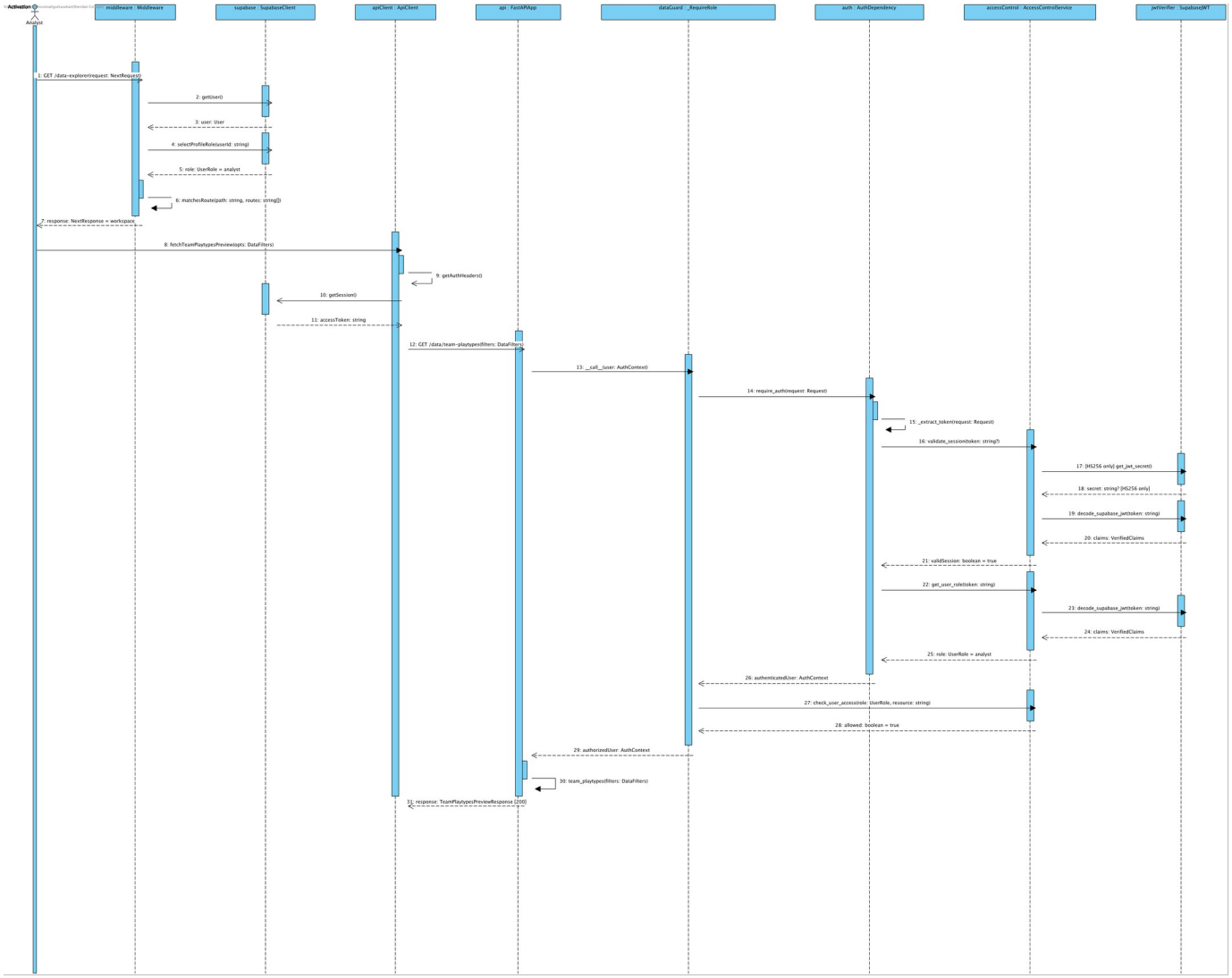
20. Collaboration Overview - Authenticate Analyst Access



21. Structural Overview - Authenticate Analyst Access



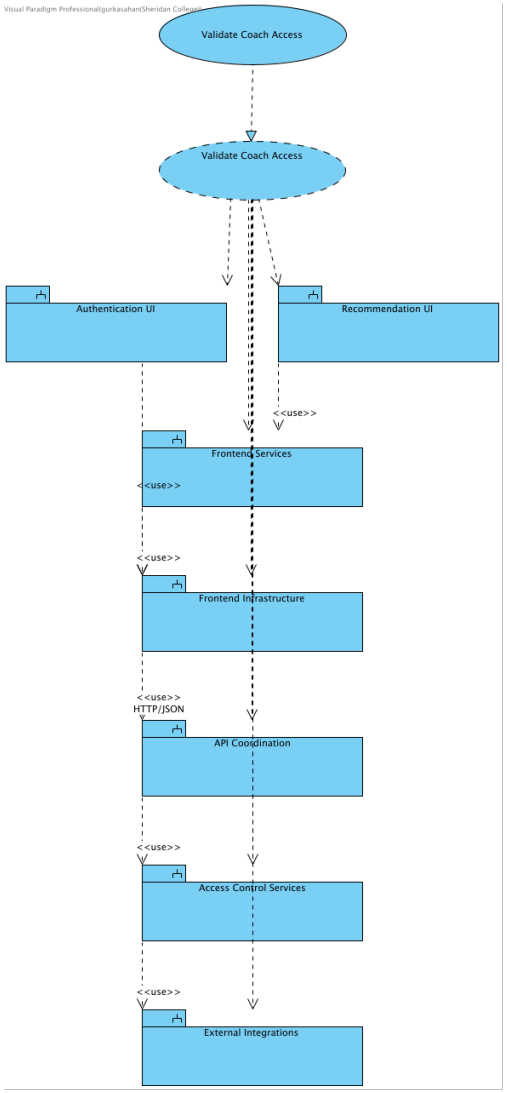
22. Behavioural Overview - Authenticate Analyst Access



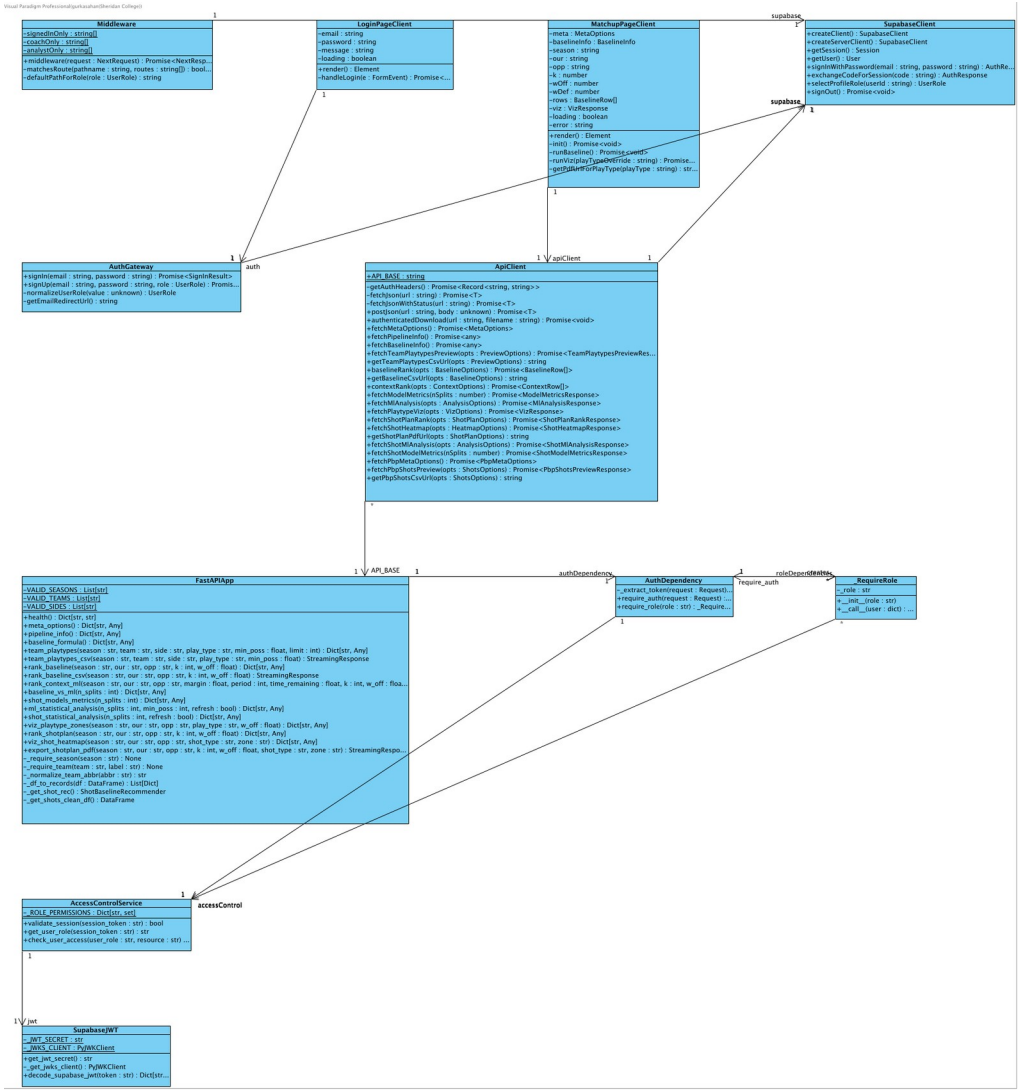
Production authorization validates ES256/RS256 JWT signatures against Supabase JWKS plus issuer, audience and expiry; verified subject identifies the user. The current protected public.profiles.role authorizes resources, never user_metadata.role. Unassigned accounts remain pending. Analyst evidence endpoints require analyst access; coach recommendations and the matchup dashboard require coach access. Shared shot views accept either approved role. Invalid tokens return 401 and forbidden roles return 403. Current explicit protected-profile authorization exchange is shown in

Detailed Sequence - Analyst Top-K Uplift (475).Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

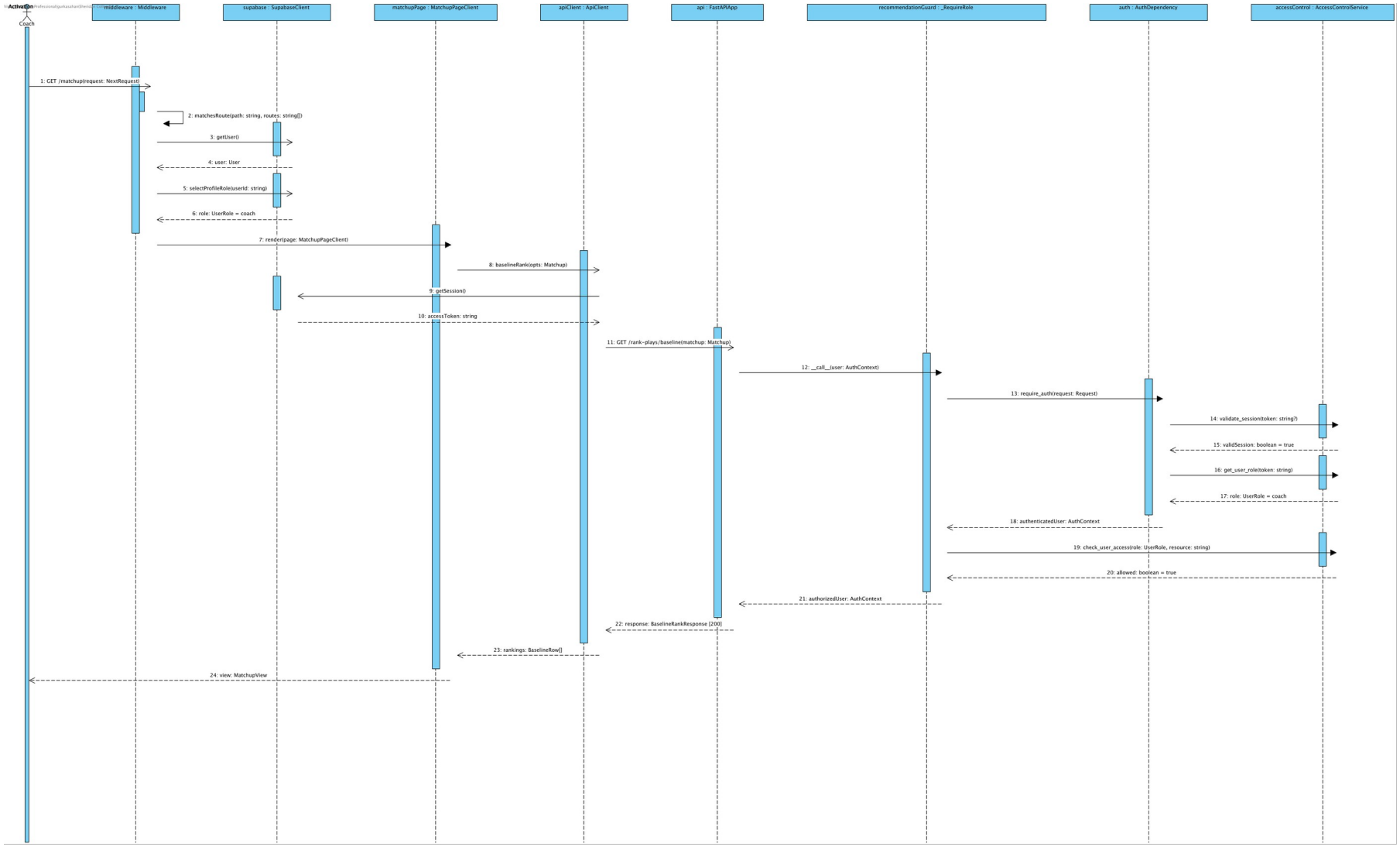
23. Collaboration Overview - Validate Coach Access



24. Structural Overview - Validate Coach Access



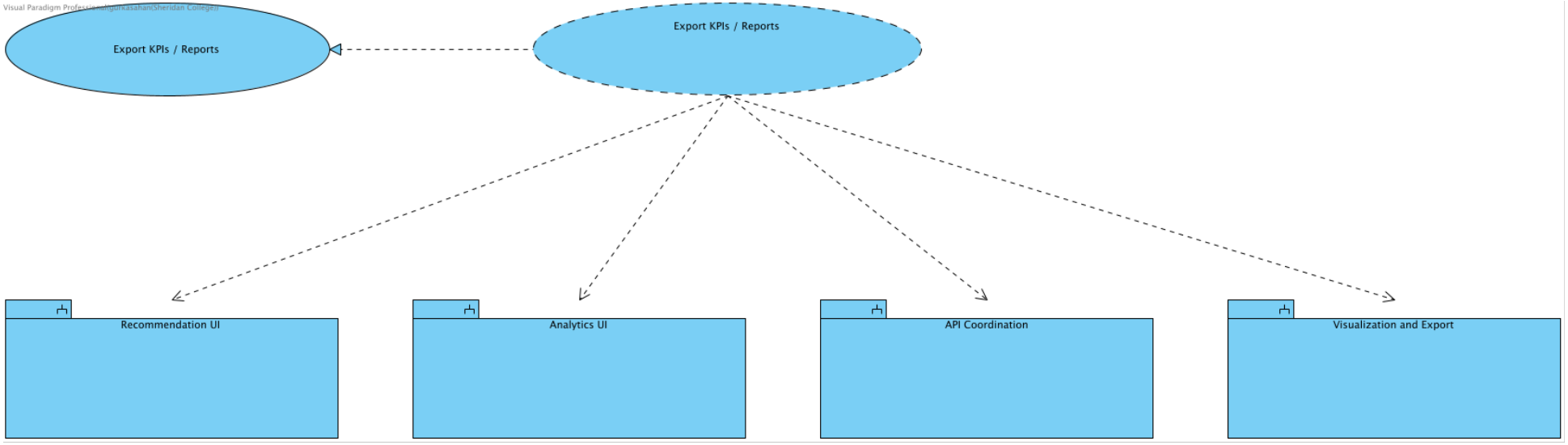
25. Behavioural Overview - Validate Coach Access



Notation: objectName : Classifier. Actor names identify user roles. Calls use operation(parameter: Type); replies use result: Type. HTTP/SDK requests use asynchronous call arrows and their completion uses a named dashed reply. Local calculations use synchronous calls. Contract names below group fields for readability; they are documentation aliases, not additional runtime classes.
Credentials = {email: string, password: string}; BaselineTables = {team_df: DataFrame, league_df: DataFrame}; UpliftInputs = {teamRows: DataFrame, rankings: DataFrame}; QualityOptions = {required_columns?: Sequence[str], key_fields?: Sequence[str], row_key?: Sequence[str], seasons?: Sequence[str], teams?: Sequence[str]}. UserRole = coach | analyst | null; null means no assigned role. VerifiedClaims contains sub, exp, aud and iss after signature/issuer/audience/expiry validation. AuthContext = {token: string, role: coach | analyst}; token values are never shown. HttpError = {status: integer, detail: string}. ApiResult<T> = successful body T or HttpError.
Matchup = {season: string, our: team code, opp: different team code, k: integer 1..10, w_off: number 0..1}. Backend w_def = 1-w_off. ContextInputs adds margin: number, period: integer 1..5, time_remaining: seconds 0..720 for regulation, 0..300 for period=5 (overtime); coaching?: CoachingConditions. Frontend maps timeRemaining to time_remaining and serializes coaching as a bounded JSON query value. CoachingConditions accepts validated need, defense_style, pace, shot_clock and supported boolean flags from coaching_conditions.py; unsupported fields return HTTP 422. The endpoint passes the validated conditions to the ranker and returns them in coaching_conditions. ContextInputs labels group the numeric situation and conditions for readability.
MetaOptions = {seasons: string[], teams: string[], teamNames: map, playTypes: string[], sides: string[], hasMlPredictions: boolean}; frontend fallback adds _fallback=true. BaselineRankResponse = {season, our_team, opp_team, k, w_off, w_def, rankings: records[]}. BaselineRow = {playType, pppPred, pppOff, pppDef, pppGap, rationale, raw}. ContextRankResponse includes matchup, context and rankings; ContextRow = {playType, finalPPP, mlPPP, baselinePPP, deltaPPP, contextLabel, rationale, raw}.
Source: middleware.ts; app/infrastructure/api-client.ts; app/infrastructure/auth.ts; backend/application/api_coordination/app.py and auth_dependency.py; backend/application/access_control_services/access_control_service.py.

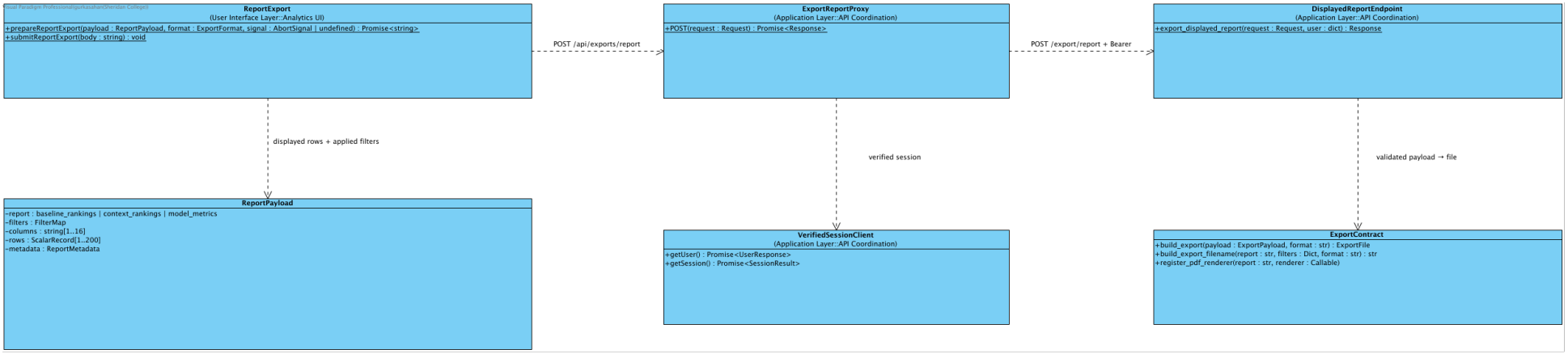
Shared result exports

26. Collaboration Overview - Export KPIs / Reports (486)



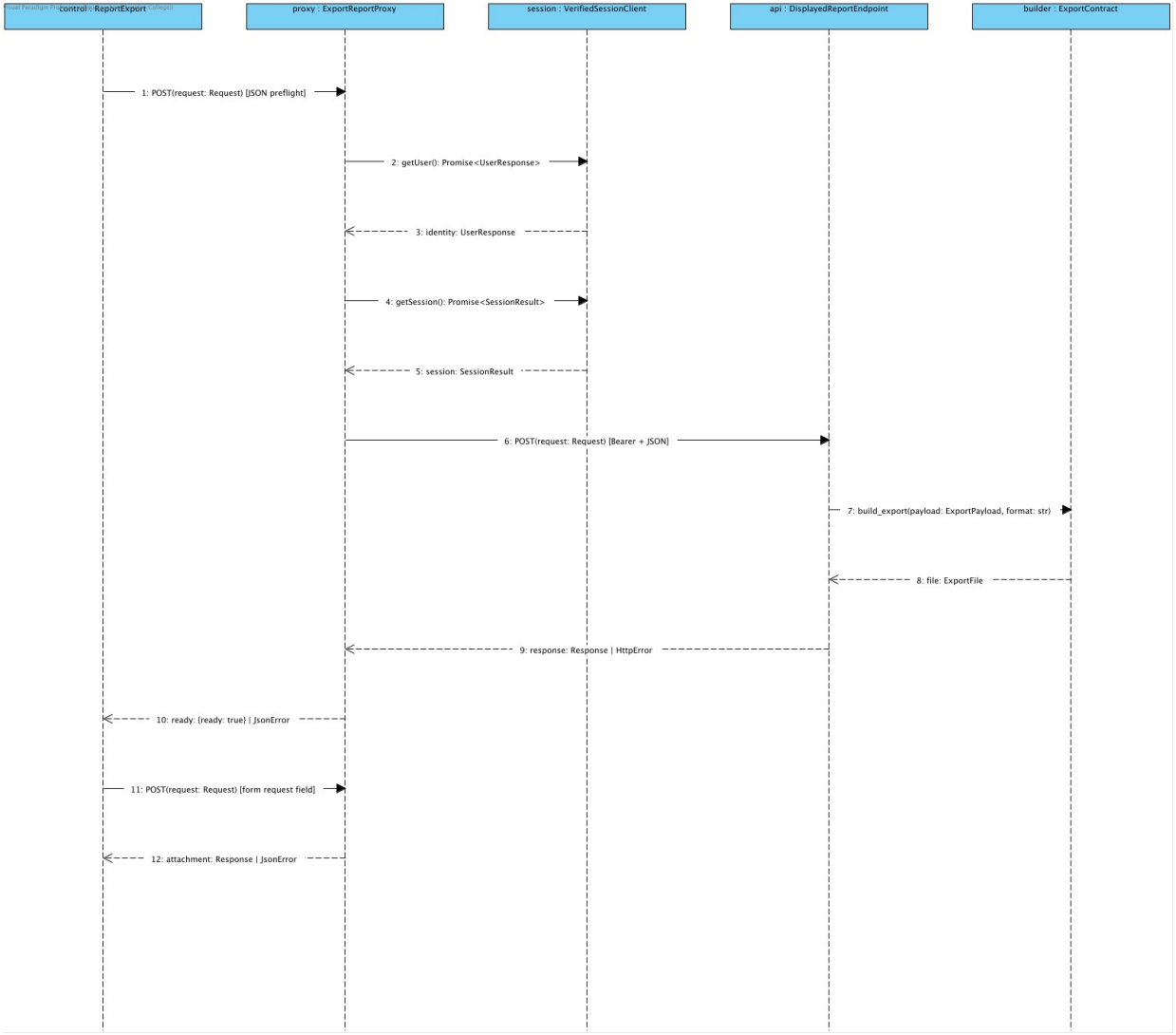
SCRUM-486. Export KPIs / Reports. This collaboration realizes the linked use case and owns its overview, structural design and detailed sequence. Modules remain in the Design Model. Implements the analyst Export KPIs / Reports use case. The same shared control is also reused by coach Matchup and Context; their existing use cases and collaboration are unchanged. Only displayed results are exported.

27. Elaboration - Shared Export Controls (486)



SCRUM-486. One Export control on Matchup, Context and Model Metrics offers CSV, JSON and PDF for the displayed result. Capture rows and the filters that produced them; disable export when results are empty, loading or no longer match the controls. This is a displayed-result snapshot, not a server recomputation. POST /api/exports/report validates the cookie session and forwards Bearer credentials to the fixed POST /export/report backend. Backend enforces report roles, 256 KiB, 200 rows, 16 columns and finite scalar cells. JSON records export_source=displayed_result_snapshot; CSV neutralizes spreadsheet formulas; PDF includes title, filters, limitations and table. Errors remain JSON without an attachment header.

28. Detailed Sequence - Export Displayed Results (486)



SCRUM-486. One Export control on Matchup, Context and Model Metrics offers CSV, JSON and PDF for the displayed result. Capture rows and the filters that produced them; disable export when results are empty, loading or no longer match the controls. This is a displayed-result snapshot, not a server recomputation. POST /api/exports/report validates the cookie session and forwards Bearer credentials to the fixed POST /export/report backend. Backend enforces report roles, 256 KiB, 200 rows, 16 columns and finite scalar cells. JSON records

export_source=displayed_result_snapshot; CSV neutralizes spreadsheet formulas; PDF includes title, filters, limitations and table. Errors remain JSON without an attachment header.
Alternatives: stale/loading/empty disables Export. Invalid session401; wrong report role403; malformed/oversized body400/413/422; upstream failure502. Preflight failure displays Retry; successful preflight starts native same-origin form download. No token is placed in a URL.
prepareReportExport(payload: ReportPayload, format: ExportFormat, signal?: AbortSignal): Promise<string> returns the encoded body. submitReportExport(body: string): void starts the native form request. Both requests run the same server checks. UserResponse and SessionResult are result envelopes, not request parameters. Errors return JSON with no attachment header. Pipeline/model values are never recomputed during export.

Source and request contracts

Compare the same regular season across all displayed teams and league references. Do not mix playoff records into regular-season totals.
Calculate team PPP as total points divided by possessions. Calculate usage as play-type possessions divided by the team's recorded offensive play-type possessions, multiplied by 100.
Calculate league values from pooled counts for the same season, side and play type. Calculate FG percentage from field goals made and attempted; weight source effective field-goal rates by attempts.
For the usage chart, subtract the same play type's league usage and PPP from the team's values. A high-efficiency, low-usage pattern is a review prompt, not proof that using that play more will improve results.
For opponent weak spots, subtract league allowed PPP from the opponent's allowed PPP. Rank the differences from highest to lowest. Highlight at most three positive differences that meet the possession cutoff.
The possession cutoff marks small samples without removing rows or changing totals. Small samples cannot receive a weak-spot highlight. The cutoff is a display rule, not a confidence interval.
Missing defensive categories remain unavailable. They must not appear as zero points allowed. The current source contains eleven offensive categories and seven defensive categories.
These comparisons use whole-season profiles. Selecting two teams does not restrict the data to games played against each other.
Only approved coaches can use the matchup dashboard. Validate the current profile role on the API as well as the page route.
Discard results that no longer match the selected filters. On an error, remove the stale chart and provide a retry action.
Keep source checks, descriptive comparisons, retrospective model diagnostics and prospective evaluation distinct. Observed-season model performance does not establish pre-game forecast accuracy.

Release boundaries

Beta: SCRUM-697 adds dedicated league comparisons; SCRUM-698 adds regular-season trends. These are planned, not part of the Alpha completion claim.
Final: SCRUM-699 adds side-by-side team profiles; SCRUM-700 adds independent possession outcome rates. These are planned, not part of the Alpha completion claim.
Alpha testing checks source totals, season isolation, aggregation denominators, ranking direction, missing categories, sample cutoffs, access control, filter changes, loading and retry behavior. Initial failures remain linked to their repair tickets. Automated checks and browser checks establish the tested behavior; they do not replace the team's recorded demonstration or a fresh domain-expert review.